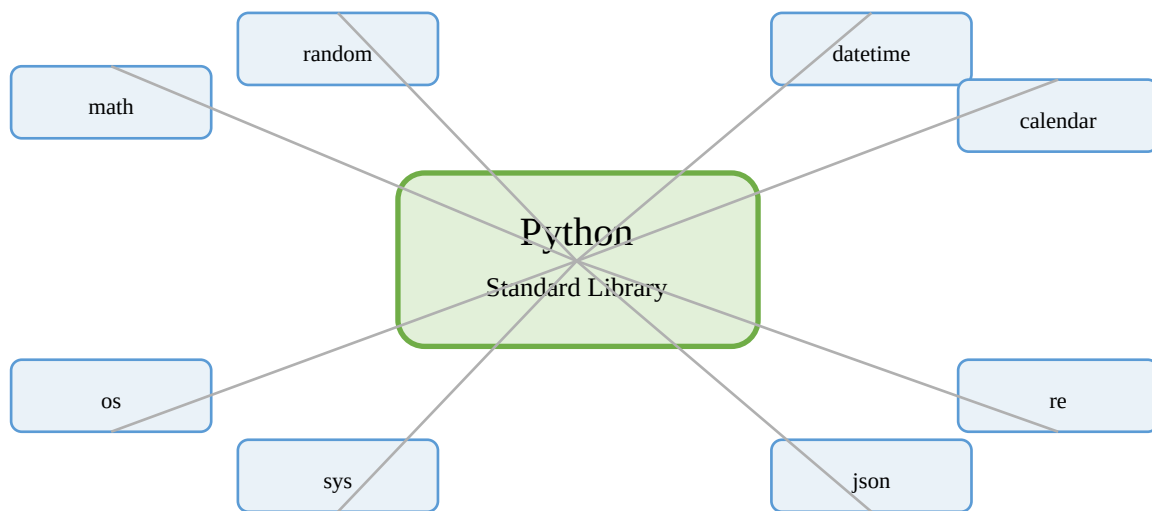


PYTHON MODULES & PACKAGES

DAY 9 • DEEP & DETAILED NOTES

Modules • Imports • Custom Modules • Packages • Python Standard Library



Concept → Syntax → Methods → Examples → Practical Use → Viva Revision

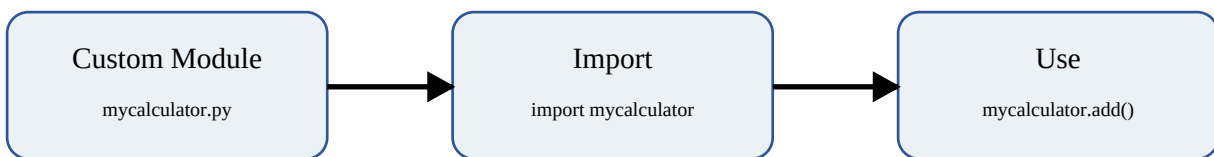
1. Module — Complete Concept

Definition: A module is a Python file ending in **.py** that contains reusable code. It may contain functions, classes, variables, and executable statements.

Real-life analogy: Think of a module as a toolbox. Instead of keeping every tool in one huge box, we keep related tools together and use the toolbox whenever needed.

Example: `math` contains mathematical functionality; a programmer can also create `mycalculator.py` for calculator functions.

Advantages: reusability, organization, maintainability, readability, separation of responsibilities, and easier testing.



Basic rule: One `.py` file can be imported as a module when Python can find it on the module search path.

2. Importing Modules — All Important Forms

Python provides several import styles. The choice depends on readability and what exactly you need.

2.1 Import the complete module

```
import math
print(math.sqrt(25))
```

Here **math** is the module and **sqrt()** is accessed through the module namespace.

2.2 Import selected names

```
from math import sqrt, factorial
print(sqrt(25))
print(factorial(5))
```

2.3 Import using an alias

```
import math as m
print(m.sqrt(25))
```

2.4 Import from a package

```
from collections import Counter
print(Counter([1, 1, 2]))
```

2.5 Import everything

```
from math import *
```

Recommendation: In real projects, prefer explicit imports. They make code easier to read and reduce accidental name conflicts.

3. Custom Modules — Step by Step

A custom module is written by you. It is useful when a group of functions/classes is needed in more than one program.

File: `mycalculator.py`

```
def add(n1, n2):
    return n1 + n2

def sub(n1, n2):
    return n1 - n2

def mul(n1, n2):
    return n1 * n2
```

File: mymain.py

```
import mycalculator

print(mycalculator.add(10, 20))
print(mycalculator.sub(20, 10))
print(mycalculator.mul(5, 4))
```

Output: **30, 10, 20**

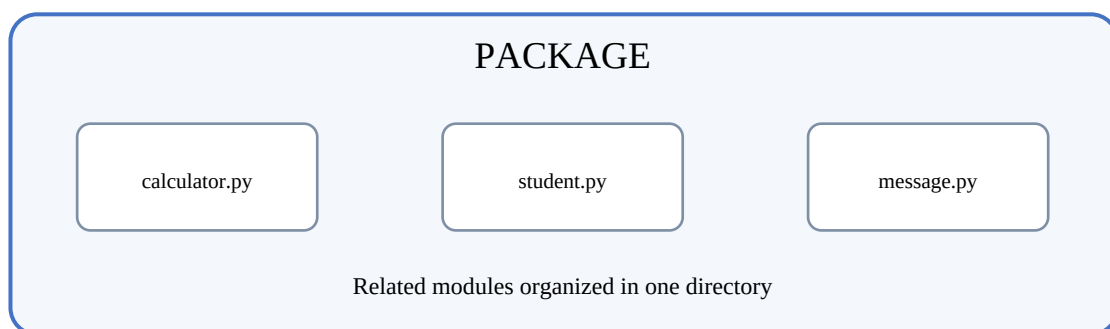
Important: return sends a value back to the caller. If a function only uses `print()` and has no return value, assigning its result to a variable gives `None`.

Example structure:

```
Day_9_Modules_packages/
├── pythonModule/
│   ├── mycalculator.py
│   └── mymain.py
```

4. Packages — Complete Concept

A package is a directory that groups related Python modules. Packages become especially useful when a project grows.



Example:

```
mypackage/
├── __init__.py
├── calculator.py
├── student.py
└── message.py
```

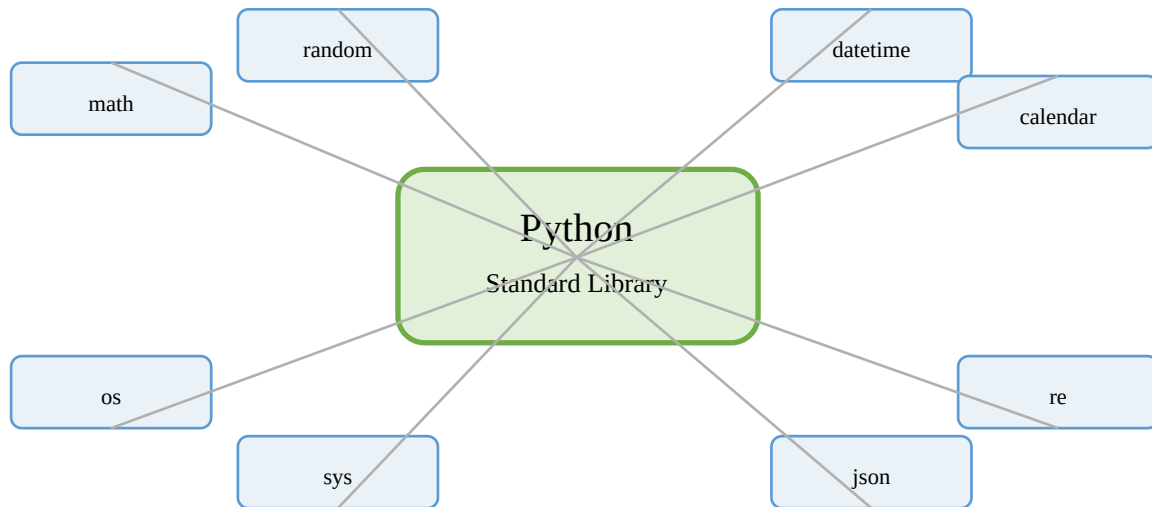
Importing from a package:

```
from mypackage.calculator import add
print(add(10, 20))
```

Note about `__init__.py`: It is commonly used in traditional Python package structures. Modern Python also supports namespace packages without it, but keeping an `__init__.py` is still common for ordinary packages and can contain package initialization code.

5. Python Standard Library — Overview

The Python Standard Library is the collection of modules distributed with Python. It covers many everyday programming needs, so separate third-party installation is not normally required for these modules.



The sections below focus on the most useful Day 9 modules and their commonly used functions/classes. The list is practical, not an exhaustive list of every function Python provides.

6. math Module

Purpose: mathematical functions, constants, rounding, powers, roots, trigonometry, logarithms, and number theory operations.

Import: `import math`

Important methods / functions / classes

Name	What it does	Typical use
<code>sqrt(x)</code>	Square root of x.	<code>math.sqrt(25) → 5.0</code>
<code>pow(x,y)</code>	x raised to y; returns a float.	<code>math.pow(2,3) → 8.0</code>
<code>factorial(n)</code>	n! for non-negative integers.	<code>math.factorial(5) → 120</code>
<code>ceil(x)</code>	Smallest integer $\geq x$.	<code>math.ceil(4.2) → 5</code>
<code>floor(x)</code>	Largest integer $\leq x$.	<code>math.floor(4.8) → 4</code>
<code>fabs(x)</code>	Absolute value as float.	<code>math.fabs(-7) → 7.0</code>
<code>gcd(a,b)</code>	Greatest common divisor.	<code>math.gcd(12,18) → 6</code>
<code>lcm(a,b)</code>	Least common multiple.	<code>math.lcm(4,6) → 12</code>
<code>isqrt(n)</code>	Integer square root.	<code>math.isqrt(26) → 5</code>
<code>sin(x), cos(x), tan(x)</code>	Trigonometric functions; x is in radians.	<code>math.sin(math.pi/2) → 1.0</code>
<code>log(x), log10(x)</code>	Natural/base-10 logarithm.	<code>math.log10(100) → 2.0</code>
<code>pi, e, tau</code>	Mathematical constants.	<code>math.pi</code>

Practical examples

```
import math

print(math.sqrt(25))
print(math.pow(2, 3))
print(math.factorial(5))
print(math.ceil(4.2))
```

```

print(math.floor(4.8))

print(math.gcd(12, 18))
print(math.lcm(4, 6))
print(math.isqrt(26))

print(math.sin(math.pi / 2))
print(math.log10(100))

```

Important: `math.add()`, `math.sub()`, `math.max()`, and `math.min()` do not exist. Python built-ins such as `max()`, `min()`, and `sum()` are called directly.

7. random Module

Purpose: random values, random selection, shuffling, sampling, and pseudo-random number generation.

Import: `import random`

Important methods / functions / classes

Name	What it does	Typical use
<code>choice(seq)</code>	Returns one randomly selected item.	<code>random.choice(names)</code>
<code>randint(a,b)</code>	Random integer N where $a \leq N \leq b$.	<code>randint(1,10)</code>
<code>randrange(start,stop,step)</code>	Random value from a range; stop excluded.	<code>randrange(1,10)</code>
<code>random()</code>	Float from 0.0 up to but not including 1.0.	<code>random.random()</code>
<code>uniform(a,b)</code>	Random float between a and b.	<code>uniform(10,20)</code>
<code>shuffle(x)</code>	Shuffles a mutable sequence in place.	<code>shuffle(names)</code>
<code>sample(pop,k)</code>	Returns k unique items from population.	<code>sample(names,2)</code>
<code>choice()</code> vs <code>sample()</code>	One item vs multiple unique items.	<code>choice(names)</code> / <code>sample(names,2)</code>

Practical examples

```

import random
names = ["Arun", "Rahul", "Amit", "Rohit", "Vivek"]
print("Random Name:", random.choice(names))

print("Integer:", random.randint(1, 100))
print("Range:", random.randrange(1, 10))

print("Float:", random.random())
print("Uniform:", random.uniform(10, 20))

random.shuffle(names)
print("Shuffled:", names)

print("Sample:", random.sample(names, 2))

```

Important: random module uses pseudo-random generation. For security-sensitive secrets such as tokens/password reset values, use the `secrets` module instead of `random`.

8. datetime Module

Purpose: date, time, datetime values, time differences, and date arithmetic.

Import: `import datetime`

Important methods / functions / classes

Name	What it does	Typical use
<code>datetime.now()</code>	Current local date and time.	<code>datetime.datetime.now()</code>
<code>date.today()</code>	Today's local date.	<code>datetime.date.today()</code>

Name	What it does	Typical use
<code>datetime(...)</code>	Creates a date-time object.	<code>datetime.datetime(2026,9,15)</code>
<code>date(...)</code>	Creates a date object.	<code>datetime.date(2026,9,15)</code>
<code>time(...)</code>	Creates a time object.	<code>datetime.time(10,30)</code>
<code>timedelta(...)</code>	Represents a duration/difference.	<code>timedelta(days=7)</code>
<code>strftime()</code>	Formats date/time as text.	<code>dt.strftime("%d-%m-%Y")</code>
<code>strptime()</code>	Parses text into date/time.	<code>datetime.datetime.strptime(...)</code>

Practical examples

```
import datetime
now = datetime.datetime.now()
print(now)
print(datetime.date.today())

from datetime import datetime, timedelta
future = datetime.now() + timedelta(days=7)
print(future)

from datetime import datetime
dt = datetime.now()
print(dt.strftime("%d-%m-%Y %H:%M:%S"))
```

Important: `datetime.datetime.now()` gives current local date/time; timezone-aware applications should consider zone information rather than treating all times as naive local times.

9. calendar Module

Purpose: displaying calendars, checking leap years, and working with calendar-related information.

Import: `import calendar`

Important methods / functions / classes

Name	What it does	Typical use
<code>month(y,m)</code>	Returns a month's calendar as text.	<code>calendar.month(2026,9)</code>
<code>calendar(y)</code>	Returns the complete year's calendar.	<code>calendar.calendar(2026)</code>
<code>isleap(y)</code>	Checks whether a year is leap year.	<code>calendar.isleap(2024)</code>
<code>monthrange(y,m)</code>	Returns weekday of first day and number of days.	<code>calendar.monthrange(2026,9)</code>
<code>weekday(y,m,d)</code>	Returns weekday number; Monday=0.	<code>calendar.weekday(2026,9,15)</code>

Practical examples

```
import calendar
print(calendar.month(2026, 9))

print(calendar.isleap(2024))

print(calendar.monthrange(2026, 9))
print(calendar.weekday(2026, 9, 15))
```

10. os Module

Purpose: operating-system interaction, directories, paths, environment information, and file-system operations.

Import: `import os`

Important methods / functions / classes

Name	What it does	Typical use
getcwd()	Returns current working directory.	os.getcwd()
listdir(path='.')	Lists entries in a directory.	os.listdir()
mkdir(path)	Creates one directory.	os.mkdir('test')
makedirs(path)	Creates nested directories.	os.makedirs('a/b/c')
rename(src,dst)	Renames/moves a path.	os.rename('a.txt','b.txt')
remove(path)	Removes a file.	os.remove('a.txt')
rmdir(path)	Removes an empty directory.	os.rmdir('test')
path.exists(path)	Checks whether path exists.	os.path.exists('test')
path.isfile(path)	Checks whether path is a file.	os.path.isfile('a.txt')
path.isdir(path)	Checks whether path is a directory.	os.path.isdir('test')

Practical examples

```
import os
print(os.getcwd())
print(os.listdir())

print(os.path.exists("pythonModule"))
print(os.path.isfile("mymain.py"))
print(os.path.isdir("pythonModule"))

# Create a directory only if it does not already exist
if not os.path.exists("test"):
    os.mkdir("test")
```

Important: Be careful with `remove()`, `rmdir()`, and `rename()`: they change the file system. `os.path.join()` is also important for building paths portably.

11. sys Module

Purpose: Python interpreter information, command-line arguments, module search paths, and process-related information.

Import: `import sys`

Important methods / functions / classes

Name	What it does	Typical use
version	Python version information.	sys.version
platform	Platform identifier.	sys.platform
argv	Command-line argument list.	sys.argv
path	Module search path list.	sys.path
exit()	Stops program execution by raising <code>SystemExit</code> .	sys.exit()
maxsize	Maximum practical integer size used by implementation.	sys.maxsize

Practical examples

```
import sys
print(sys.version)
print(sys.platform)

import sys
print(sys.argv)

import sys
print(sys.path)
```

Important: `sys.argv[0]` is normally the script name; later entries contain command-line arguments.

12. json Module

Purpose: encoding Python objects as JSON and decoding JSON into Python objects. Common in APIs, configuration, and data exchange.

Import: `import json`

Important methods / functions / classes

Name	What it does	Typical use
<code>loads(s)</code>	JSON string → Python object.	<code>json.loads('{\"a\":1}')</code>
<code>dumps(obj)</code>	Python object → JSON string.	<code>json.dumps({'a':1})</code>
<code>load(file)</code>	Read JSON from a file object.	<code>json.load(f)</code>
<code>dump(obj,file)</code>	Write JSON to a file object.	<code>json.dump(data,f)</code>

Practical examples

```
import json
data = '{"name":"Arun","age":23}'
obj = json.loads(data)
print(obj["name"])

data = {"name":"Arun", "skills":["Python","SQL"]}
text = json.dumps(data, indent=2)
print(text)

# File example
# with open("data.json", "w") as f:
#     json.dump(data, f, indent=2)
```

Important: Remember: `loads/dumps` work with strings; `load/dump` work with file objects.

13. re Module

Purpose: regular expressions for searching, matching, extracting, splitting, and replacing text patterns.

Import: `import re`

Important methods / functions / classes

Name	What it does	Typical use
<code>search(pattern,string)</code>	Finds first match anywhere.	<code>re.search(r'\d+', text)</code>
<code>match(pattern,string)</code>	Matches only at the beginning.	<code>re.match(r'Hello', text)</code>
<code>findall(pattern,string)</code>	Returns all matching strings.	<code>re.findall(r'\d+', text)</code>
<code>finditer(pattern,string)</code>	Returns match objects iteratively.	<code>re.finditer(...)</code>
<code>sub(pattern,repl,string)</code>	Replaces matching text.	<code>re.sub(r'\d+', 'X', text)</code>
<code>split(pattern,string)</code>	Splits text using a pattern.	<code>re.split(r'\s+', text)</code>

Practical examples

```
import re
text = "My marks are 85 and 92"
print(re.findall(r"\d+", text))

text = "abc123xyz"
match = re.search(r"\d+", text)
if match:
    print(match.group())
```

```
text = "Python 123 Java 456"
print(re.sub(r"\d+", "NUMBER", text))
```

Important: Raw strings such as `r'\d+'` make regex backslashes easier to write. Common patterns: `\d` digit, `\w` word character, `\s` whitespace, `+` one or more, `*` zero or more, `?` optional.

14. statistics Module

Purpose: common statistical calculations such as mean, median, mode, variance, and standard deviation.

Import: `import statistics`

Important methods / functions / classes

Name	What it does	Typical use
<code>mean(data)</code>	Arithmetic average.	<code>statistics.mean([10,20,30])</code>
<code>median(data)</code>	Middle value after sorting.	<code>statistics.median([10,20,30])</code>
<code>mode(data)</code>	Most common value.	<code>statistics.mode([1,1,2])</code>
<code>multimode(data)</code>	All modes.	<code>statistics.multimode([1,1,2,2])</code>
<code>variance(data)</code>	Sample variance.	<code>statistics.variance(data)</code>
<code>stdev(data)</code>	Sample standard deviation.	<code>statistics.stdev(data)</code>

Practical examples

```
import statistics
marks = [70, 80, 90, 60, 100]
print("Mean:", statistics.mean(marks))
print("Median:", statistics.median(marks))

data = [10, 10, 20, 30, 10]
print(statistics.mode(data))
print(statistics.multimode(data))
```

15. collections Module

Purpose: specialized container data types that extend normal Python collection patterns.

Import: `from collections import ...`

Important methods / functions / classes

Name	What it does	Typical use
Counter	Counts hashable objects.	<code>Counter(['a','a','b'])</code>
defaultdict	Dictionary with a default factory.	<code>defaultdict(list)</code>
deque	Fast appends/pops from both ends.	<code>deque([1,2,3])</code>
namedtuple	Tuple subclass with named fields.	<code>namedtuple('Point','x y')</code>

Practical examples

```
from collections import Counter
names = ["Arun", "Amit", "Arun", "Rahul", "Amit", "Arun"]
print(Counter(names))

from collections import deque
q = deque([1, 2, 3])
q.append(4)
q.popleft()
print(q)

from collections import defaultdict
groups = defaultdict(list)
groups['A'].append('Arun')
```

```
print(groups)
```

Important: Counter is especially useful for frequency counting; deque is useful for queue-like operations.

16. time Module

Purpose: wall-clock and sleep-related operations, timestamps, and basic timing.

Import: `import time`

Important methods / functions / classes

Name	What it does	Typical use
<code>time()</code>	Returns seconds since the Unix epoch as a float.	<code>time.time()</code>
<code>sleep(sec)</code>	Pauses execution for a number of seconds.	<code>time.sleep(2)</code>
<code>ctime([secs])</code>	Converts timestamp to readable local time.	<code>time.ctime()</code>
<code>perf_counter()</code>	High-resolution timer for measuring durations.	<code>time.perf_counter()</code>

Practical examples

```
import time
print("Hello")
time.sleep(2)
print("Python")

import time
start = time.perf_counter()
# code to measure
end = time.perf_counter()
print("Elapsed:", end - start)
```

Important: Use `perf_counter()` for measuring elapsed execution time. For calendar dates/times, prefer `datetime`.

17. Built-in Functions vs Module Functions

Not every useful Python function belongs to a module. Python also provides built-in functions that can be called directly.

```
print(max(10, 20))
print(min(10, 20))
print(sum([10, 20, 30]))
print(len([10, 20, 30]))
```

These are different from module-qualified calls such as `math.sqrt(25)`. Knowing this distinction prevents errors like `math.max()`.

18. Common Import Errors & Fixes

ModuleNotFoundError: Python cannot find the requested module. Check spelling, file location, virtual environment, and Python path.

AttributeError: The module exists, but the requested attribute/function does not. Example: `math.add()`.

NameError: A name/function was used without being imported or defined.

Practical tip: If a custom module is in the same folder as your main file, a simple `import module_name` usually works.

19. Complete Day 9 Project Structure

```
Day_9_Modules_packages/
■■■ pythonModule/
■■■ mycalculator.py # Custom module
■■■ mymain.py # Uses custom module
■■■ mathmodule.py # math practice
■■■ randommodule.py # random practice
■■■ datetimemodule.py # datetime practice
■■■ calendarmodule.py # calendar practice
■■■ osmodule.py # os practice
■■■ sysmodule.py # sys practice
■■■ jsonmodule.py # json practice
■■■ remodule.py # re practice
■■■ statisticmodule.py # statistics practice
■■■ collectionsmodule.py # collections practice
■■■ timemodule.py # time practice
```

20. Quick Revision Sheet

Module	Remember these	Main purpose
math	sqrt, pow, factorial, ceil, floor, gcd, lcm, pi	Math
random	choice, randint, randrange, shuffle, sample, random, uniform	Random values
datetime	now, today, timedelta, strftime, strptime	Date & time
calendar	month, calendar, isleap, monthrange, weekday	Calendar
os	getcwd, listdir, mkdir, path.exists, path.isfile, path.isdir	OS/files
sys	version, platform, argv, path, exit	Interpreter/system
json	loads, dumps, load, dump	JSON
re	search, match, findall, sub, split	Patterns/text
statistics	mean, median, mode, variance, stdev	Statistics
collections	Counter, defaultdict, deque, namedtuple	Containers
time	time, sleep, ctime, perf_counter	Timing

21. Viva / Interview Questions

1. **What is a module?** A reusable Python file containing code such as functions, classes, variables, and statements.
2. **What is a package?** A directory that organizes related Python modules.
3. **How do you import a module?** `import module_name`
4. **Difference between `import math` and `from math import sqrt`?** The first imports the module namespace; the second imports only `sqrt` into the current namespace.
5. **`choice()` vs `sample()`?** `choice` selects one item; `sample` returns multiple unique items.
6. **`randint()` vs `randrange()`?** `randint` includes both endpoints; `randrange` excludes the stop value.
7. **What does `shuffle()` do?** It changes the order of elements in a mutable sequence in place.
8. **`loads()` vs `load()`?** `loads` reads JSON from a string; `load` reads JSON from a file object.
9. **What is the Standard Library?** Modules distributed with Python that provide common functionality.
10. **Why use custom modules?** To reuse and organize project-specific code.

22. Final Concept Map

```
PYTHON CODE
|
+--> MODULE
| |
| +--> Functions / Classes / Variables
| +--> import module
|
+--> PACKAGE
| |
| +--> Related Modules
|
+--> STANDARD LIBRARY
|
+--> math -> Mathematics
+--> random -> Random values
+--> datetime -> Date/time
+--> calendar -> Calendar
+--> os -> OS/files
+--> sys -> Interpreter/system
+--> json -> JSON data
+--> re -> Patterns
+--> statistics-> Statistics
+--> collections-> Containers
+--> time -> Timing
```

DAY 9 CORE IDEA: Write reusable code once, organize it into modules/packages, and import only what your program needs.